

Fractal de Mandelbrot com C e Python

Trabalho de Implementação — Duas Linguagens

Ulisses Gilioli Francescatto Júnior

UFPel - Julho de 2026

1. Aplicação escolhida

A aplicação implementada gera o **fractal de Mandelbrot**: uma imagem obtida a partir do comportamento da iteração $z_{n+1} = z_n^2 + c$ no plano complexo. Para cada pixel da tela, associa-se um número complexo c ; a cor do pixel depende de quantas iterações são necessárias até que $|z|$ ultrapasse o valor 2 (indicando que o ponto diverge). Pontos que não divergem até um número máximo de iterações são considerados pertencentes ao conjunto e pintados de preto.

A aplicação é interativa: o usuário pode clicar com o botão esquerdo do mouse para dar zoom em uma região da imagem, ou com o botão direito para afastar o zoom. A cada clique, uma nova região do plano complexo é recalculada e redesenhada na tela.

2. Divisão entre as linguagens

O trabalho exige o uso conjunto de duas linguagens com papéis distintos: uma responsável pela interface com o usuário, outra pelo serviço de cálculo. A divisão adotada foi:

Linguagem	Arquivo	Responsabilidade
C	<code>mandelbrot.c</code>	Cálculo numérico do fractal: para cada pixel, itera a fórmula de Mandelbrot até a divergência ou o limite máximo de iterações.
Python	<code>interface.py</code>	Interface gráfica: janela, captura de cliques do mouse, coloração da imagem e controle do zoom in / zoom out.

O código em C foi escolhido para o cálculo por ser uma linguagem compilada e de baixo nível, adequada para a natureza repetitiva e intensiva do cálculo do fractal (um laço duplo sobre todos os pixels da tela, com uma iteração interna por pixel). O Python foi escolhido para a interface por oferecer bibliotecas de alto nível prontas para criação de janelas e captura de eventos (**Pygame**), além de manipulação eficiente de arrays (**NumPy**), reduzindo significativamente a quantidade de código necessário para a parte gráfica.

3. Método de interface entre as linguagens

A comunicação entre Python e C é feita através do módulo **ctypes**, da biblioteca padrão do Python, que permite carregar uma biblioteca compartilhada (**.so**) e chamar suas funções diretamente, sem necessidade de processos externos, sockets ou arquivos intermediários.

3.1. Compilação como biblioteca compartilhada

O arquivo **mandelbrot.c** é compilado com a *flag* **-shared -fPIC**, gerando o arquivo **libmandelbrot.so**. Diferentemente de um executável comum, uma biblioteca compartilhada não é executada diretamente: seus símbolos (funções) ficam disponíveis para serem carregados e chamados por outro programa em tempo de execução — no caso, o interpretador Python.

3.2. Carregamento da biblioteca em Python

No **interface.py**, a biblioteca é carregada com:

```
lib = ctypes.CDLL("./libmandelbrot.so")
```

Em seguida, os tipos dos parâmetros da função **calcular_mandelbrot** são declarados explicitamente por meio do atributo **argtypes**. Essa etapa é necessária porque o Python é uma linguagem dinamicamente tipada e o C é estaticamente tipado: o **ctypes** precisa saber exatamente qual tipo C (**c_int**, **c_double**, ponteiro etc.) corresponde a cada argumento para montar corretamente a chamada em baixo nível e converter os tipos entre as duas linguagens.

3.3. Compartilhamento de memória via buffer

O ponto central do método de interface é a passagem de um **buffer de memória** entre as duas linguagens, evitando cópias desnecessárias de dados:

1. O Python aloca um array com **NumPy** (**np.zeros(largura * altura, dtype=np.int32)**);
2. Obtém um ponteiro C para esse mesmo array com **matriz_resultado.ctypes.data_as(ctypes.POINTER(ctypes.c_int32))**;
3. Passa esse ponteiro como argumento na chamada da função C;
4. A função **calcular_mandelbrot**, do lado do C, escreve diretamente nesse endereço de memória, uma posição por pixel (**resultado[tela_y * largura + tela_x] = iteracao**);
5. Ao retornar da chamada, o array NumPy already contém os valores calculados pelo C, sem que nenhum dado precise ser serializado, copiado ou transmitido por outro meio.

Esse é o mecanismo que caracteriza o uso conjunto das duas linguagens: o Python delega inteiramente o cálculo pesado ao C, e o C devolve o resultado escrevendo direto na região de memória que o Python já havia reservado.

3.4. Uso do resultado pela interface

Após a chamada, o Python remodela o array unidimensional para o formato de imagem (**altura × largura**) com **reshape**, converte cada valor de iteração em uma cor RGB, e desenha o resultado em uma janela **Pygame**. Cliques do mouse recalculam os limites **x_min**, **x_max**, **y_min**, **y_max** do plano complexo e disparam uma nova chamada à função em C, repetindo o ciclo.

4. Estrutura do repositório

.
README.md - README com informações gerais
Makefile - Make para criar o executavel
mandelbrot.c - Parte em C que realiza os caculos
interface.py - Parte em Python que cria a interface
documentacao.pdf - Este documento

5. Como compilar e executar

A compilação gera a biblioteca compartilhada `libmandelbrot.so` a partir de `mandelbrot.c`. A execução roda `interface.py`, que carrega essa biblioteca. Ambos os passos estão automatizados no `Makefile` (ver `README.md` para detalhes de uso e dependências).